Perlustra, the codebase health audit — Sample audit · eShopOnWeb

Prepared by Mervenor · 2026-10-29 · Version 1.0 · Sample audit: a public example, not prepared for a client

Scope: repository `dotnet-architecture/eShopOnWeb` at commit `4da8212117e87d808d4bbc7da6286fd2147ce606` (2025-01-13) · environments: none, source only · assessed 28–29 October 2026 · access: read-only public source code

> **About this sample.** eShopOnWeb is a reference and learning application published by Microsoft under the MIT License. Its README says it "is not meant to be an eCommerce reference application", and the repository has been archived since 13 January 2025. Many of the items below are deliberate teaching simplifications, and several are marked `TODO` in the code itself. We review it as if it were a production application you had just inherited, because that is the situation our clients are in. This is a **desk review**: we read the code and configuration and checked public package metadata. We did not build, restore, run or test anything. Section 8 lists what that leaves unverified.

---

1. Executive summary

Overall rating: Red. As inherited, this application should not run in production. The good news is that the codebase is small (about 7,300 non-blank lines of C# and Razor) and cleanly layered, so the fixes are contained.

The three findings that matter most

1. **Customers can read each other's orders** (F-01) — Critical — any signed-in shopper can open another customer's order, including the shipping address, by changing a number in the address bar.
2. **Administrator access rests on values published in the source code** (F-02, F-03) — Critical — the API's token-signing key and the seeded administrator password are both in the public repository, so anyone can take control of the catalogue and the admin console.
3. **The runtime goes out of support on 10 November 2026** (F-04, F-05) — High — security fixes for .NET 8 stop in 12 days. The framework packages are already 31 patch releases behind, and some carry published High-severity advisories.

Our recommendation: Stabilise with a **Firmora sprint**, then Upgrade with a **Renascia sprint**. Close the three Critical gaps and the database exposure first, because they can be exploited today. Then move to .NET 10, because .NET 8 patches end on 10 November. If you want both at once, the upgrade can run alongside the Firmora sprint.

Recommended next step: a Firmora sprint of **10 engineering days** (two weeks for one engineer), plus up to 1.5 days of contingency. Scope, exclusions and assumptions are in section 7.

Overall health: 0 of 10 dimensions Green · 5 Amber · 5 Red (section 3). The full roadmap is **43.5 engineering days**, plus up to 5.5 days of contingency (section 6).

---

2. System overview

- **Purpose and users:** an online shop for branded merchandise. Shoppers browse a catalogue, keep a basket, sign in, check out and view past orders. Administrators manage the catalogue in a separate console. Checkout records a fixed placeholder address and takes no payment (`src/Web/Pages/Basket/Checkout.cshtml.cs:57`). The upstream README says the sample leaves out features a real shop needs (`README.md:27`).
- **Architecture:**

```
Browser ──► Web (ASP.NET Core MVC + Razor Pages, cookie sign-in)
  │            ├─ storefront, basket, checkout, orders, account pages
  │            └─ hosts the Blazor WebAssembly admin client (/admin)
  └─(admin client, JWT)──► PublicApi (ASP.NET Core minimal API endpoints)
Web and PublicApi ──► ApplicationCore (entities, specifications, services)
                  ──► Infrastructure (EF Core) ──► SQL Server: catalogue DB, identity DB
Infrastructure as code (Bicep + Azure Developer CLI): App Service, two Azure SQL servers, Key Vault
```

- **Stack and versions (as found in the code):** .NET 8 (`net8.0`) across all ten projects; ASP.NET Core, EF Core and Blazor WebAssembly packages at 8.0.2; SQL Server via EF Core; xUnit 2.7.0 and MSTest 3.2.2; jQuery and Bootstrap at different versions in development and production (F-12); GitHub Actions for build and test; Bicep with the Azure Developer CLI for hosting. Size: 5,253 non-blank lines of C# and 2,036 of Razor in `src/`, plus 1,689 lines of tests (evidence/01).
- **History as told to us:** none; this is a sample with no kick-off interview. From the repository: first commit 2017-03-21; 825 commits; the last change to application code or packages was 2024-04-01; archived 2025-01-13 (evidence/00).
- **Pain reported at kick-off:** not applicable to a sample.

3. Scorecard

#	Dimension	Rating	Evidence
1	Build and run reproducibility: a clean checkout builds; setup is documented	Amber	README documents four ways to run it. But `global.json` asks for `8.0.x`, which the SDK's documentation does not accept; there are no lock files; CI allows prerelease SDKs; the local database image was retired in 2025 (F-20)
2	Dependency and runtime currency: end-of-life dates, known vulnerabilities	Red	.NET 8 ends support on 10 Nov 2026. 14 packages pinned at 8.0.2 (latest 8.0.33). 3 direct and 9 transitive packages carry advisories. Production pages load front-end libraries that reached end of life years ago (F-04, F-05, F-12, F-14)
3	Architecture and code structure: boundaries, coupling, dead code	Amber	Clear core / infrastructure / web / API layers, specifications, central package management. However, the core depends on a UI project, dead code has accumulated, and a few error-prone patterns recur (F-21, F-22)
4	Data layer: schema, migrations, integrity, backups	Red	Basket lines have no foreign key to products. Migrations run at application start-up. The database is open to almost any address and the app user is `db_owner`. Backups are not defined in code (F-07, F-08)
5	Tests and quality gates: coverage, CI, linting, types	Amber	62 test methods, including API authorisation tests. But every database test uses the in-memory provider, one unit test asserts the F-01 behaviour, and CI is build-and-test only on outdated actions (F-16)
6	Deployment and environments: staging, rollback, infrastructure as code	Red	Infrastructure as code exists but deploys only the Web app, not the API the admin console needs. Deployment is manual from a workstation, with no staging or rollback; the B1 plan offers no deployment slots (F-09)
7	Observability: logging, error tracking, monitoring, alerts	Red	Console and file logs only; Application Insights is not wired; no alerts. Health checks pass only while a product called ".NET Bot Black Sweatshirt" exists (F-10)
8	Security hygiene: authentication, secrets, input validation, access control	Red	Order detail lacks an ownership check; the signing key and admin password are in source; the sign-in cookie loses `HttpOnly`; error text leaks to callers (F-01, F-02, F-03, F-06, F-11, F-13, F-23)
9	Performance and cost: hot spots, bundle size, hosting spend	Amber	A one-second sleep on every catalogue API call; unbounded page sizes; 2.7 MB of storefront imagery. The app assumes a single, never-restarted instance; anonymous visits create database rows (F-15, F-18, F-19)
10	Documentation and knowledge risk: runbooks, decision records, what only one person knows	Amber	Good local-setup README. No runbook, decision records or security policy; upstream is archived, so every future fix is yours (F-24)

Speed, accessibility and search health (checked alongside the code review)

Check	Result	Evidence
Page speed on mobile (Core Web Vitals, lab)	**Not measured**: lab scores need the site running. Static indicators: a 730 KB banner PNG used as a CSS background on catalogue pages; 12 product PNGs totalling 2.0 MB; no response compression or output caching; one-second server delay on the admin catalogue API	evidence/01, evidence/07, F-15; desk review, 2026-10-28
Accessibility (WCAG 2.2 automated checks)	**Not run**: automated tools need rendered pages. Reading the markup: 19 of 20 `<img>` without `alt`, an unnamed image submit button, no `lang` on either layout, zoom disabled on the admin page, a hover-only account menu, unlabelled inputs, click-only table rows (F-17)	evidence/07 static scans; desk review, 2026-10-28
Search health (crawl, indexability, redirects, structured data)	**Not crawled**. From the views: no meta descriptions, generic titles ("Catalog - Microsoft.eShopOnWeb"), no `robots.txt` or sitemap, no structured data for products	`src/Web/Views/Shared/_Layout.cshtml:3-24`; `src/Web/wwwroot` listing; desk review, 2026-10-28

3.1 Security basics against the OWASP Top 10:2025

OWASP category	What we found	Findings
A01 Broken Access Control	Order detail returns any customer's order; a health check fetches a URL taken from the request's Host header (server-side request forgery is mapped here)	F-01, F-23
A02 Security Misconfiguration	Database open to the internet with an owner-level app user; developer tools and Swagger outside development; cookie hardening overridden	F-08, F-11, F-06
A03 Software Supply Chain Failures	Packages with published advisories; end-of-life CDN scripts without integrity checks; a deployment script that downloads a tool from the internet; licence changes on the fix path	F-05, F-12, F-08, F-14
A04 Cryptographic Failures	Token-signing key and a data-protection key committed to source	F-02
A05 Injection	No raw SQL (`FromSql`/`ExecuteSql`) and no `Html.Raw` or `MarkupString` found; EF Core parameterises queries and Razor encodes output. **No finding**	evidence/07
A06 Insecure Design	Deleting a product breaks dependent baskets; demo credentials printed on the login page; health tied to a product name	F-07, F-03, F-10
A07 Authentication Failures	Seeded administrator with a published password; no e-mail confirmation; six-character passwords; seven-day tokens; no sign-in rate limiting	F-03, F-13
A08 Software or Data Integrity Failures	Third-party scripts loaded without Subresource Integrity; JWT issuer and audience not validated	F-12, F-02
A09 Security Logging and Alerting Failures	No telemetry or alerts; failed sign-ins and errors go unnoticed	F-10
A10 Mishandling of Exceptional Conditions	Raw exception messages returned by the API; `First(...)` throws on missing products; a seeding retry that rethrows after success	F-11, F-07, F-22

3.2 What is already in good shape

We want to be fair to the code. Much of it is well made, and the plan below builds on that rather than replacing it.

- Clear layering: domain logic in `ApplicationCore`, persistence in `Infrastructure`, and thin pages and endpoints. Queries use named specifications, which keeps data access testable.
- Central package management (`Directory.Packages.props`) makes the upgrade a single-file change for versions. Dependabot was configured.
- The production configuration path reads connection strings from Key Vault through a managed identity (`src/Web/Program.cs:29-42`, `infra/main.bicep:59-63`), and App Service is set to HTTPS-only with TLS 1.2 minimum.
- Adding to the basket takes the price from the database, not from the form (`src/Web/Pages/Basket/Index.cshtml.cs:40-48`), so shoppers cannot set their own prices.
- Account-management actions validate anti-forgery tokens (`src/Web/Controllers/ManageController.cs`, 10 actions). The API has tests showing a non-admin token is refused (`tests/PublicApiIntegrationTests/CatalogItemEndpoints/CreateCatalogItemEndpointTest.cs:27-32`).
- The README gets a new developer running in four different ways (LocalDB, Docker, dev container, Azure Developer CLI).

3.3 Hosting and cost drivers (from the templates; no bills seen)

- **App Service plan B1** with Always On (`infra/main.bicep:128-130`, `appservice.bicep:24`): one Basic-tier plan. Basic has no deployment slots, so adding staging (F-09) means moving up a tier.
- **Two Azure SQL logical servers, one database each** (`main.bicep:77-106`), with **no database tier set** (`sqlserver.bicep:28-31`). The tier, and so the largest running cost, is whatever the platform default is at deployment time. Two databases could share one server, or one database with two schemas.
- **The PublicApi is not provisioned** (F-09). Hosting it adds an app, on the same plan or another.
- **Verbose file logging** with detailed errors (`appservice.bicep:77-88`): disk and noise, with no telemetry value.
- **Images are served by the app**, with no CDN and no modern formats (F-15). Bandwidth grows with traffic.
- **Deployment-time resources:** the SQL user script runs as an Azure deployment script (`sqlserver.bicep:45-97`), which the platform runs in temporary container and storage resources.

---

4. Detailed findings

Findings are ordered by severity. Severity: **Critical** (production or data at risk now) · **High** (blocks the roadmap or will fail within months) · **Medium** (a drag on cost or quality) · **Low** (hygiene). Line numbers refer to commit `4da8212`; `evidence/02-code-excerpts.md` reproduces each cited block.

F-01 · Any signed-in customer can open any customer's order · Severity: Critical · Dimension: 8

- **What we found:** The order-detail action passes both the signed-in user's name and the order number to its query, but the query handler uses only the order number. Order numbers are sequential integers. The "My orders" list is correctly filtered by buyer; the detail page is not.
- **Evidence:** `src/Web/Controllers/OrderController.cs:31-35` sends `GetOrderDetails(User.Identity.Name, orderId)`. `src/Web/Features/OrderDetails/GetOrderDetailsHandler.cs:21-22` builds `OrderWithItemsByIdSpec(request.OrderId)` and never reads `request.UserName`. `src/ApplicationCore/Specifications/OrderWithItemsByIdSpec.cs:10-13` filters on `order.Id` only, whereas `CustomerOrdersSpecification.cs:10` filters on `BuyerId`. Order ids are identity columns (`CatalogContextModelSnapshot.cs:154-158`). The unit test `tests/UnitTests/MediatorHandlers/OrdersTests/GetOrderDetails.cs:22-37` creates an order for buyer `buyerId`, requests it as `SomeUserName` and asserts that a result comes back, so the test encodes the gap. OWASP A01:2025 (CWE-639).
- **Why it matters:** any customer can read other customers' orders (products, totals and shipping address) by changing the number in the address bar. That is a personal-data breach, notifiable under data-protection law in many jurisdictions.
- **What to do:** filter by buyer and id in one specification and return "not found" when nothing matches. Replace the unit test with one that expects no result for another user, and add a functional test with two accounts. Check access logs for sequential requests to the order-detail address.
- **Effort:** 0.5 days · **In the recommended sprint:** yes

F-02 · API administrator tokens can be forged from the public source · Severity: Critical · Dimension: 8

- **What we found:** The PublicApi validates bearer tokens with a symmetric key held as a string constant in source, and it validates neither issuer nor audience. Anyone who reads the repository can sign a token carrying the `Administrators` role. The repository's own test helper does exactly that, without a password. Admin-only endpoints create, update and delete catalogue items. Other credentials are committed too: an ASP.NET Core data-protection master key, stored unencrypted (it expired on 2022-03-01 and no code references it), and a database `sa` password in the local compose and Docker settings.
- **Evidence:** `src/ApplicationCore/Constants/AuthorizationConstants.cs:10-11` (`JWT_SECRET_KEY`, under "TODO: Change this to an environment variable"). `src/PublicApi/Program.cs:54-69`: key from the constant; `ValidateIssuer = false`, `ValidateAudience = false`, `RequireHttpsMetadata = false`. `src/Infrastructure/Identity/IdentityTokenClaimService.cs:26, 37-41` signs with the same constant. `tests/PublicApiIntegrationTests/ApiTokenHelper.cs:13-47` mints an admin token from it. The role checks sit in `CreateCatalogItemEndpoint.cs:29-30`, `UpdateCatalogItemEndpoint.cs:27-28` and `DeleteCatalogItemEndpoint.cs:20-21`. Also `src/Web/key-768c1632-cf7b-41a9-bb7a-bff228ae8fba.xml:5,12`, `docker-compose.yml:23`, `src/Web/appsettings.Docker.json:3-4`, `src/PublicApi/appsettings.Docker.json:3-4`. Values are redacted in our excerpts. OWASP A04:2025, A08:2025.
- **Why it matters:** wherever the API is deployed (the admin console needs it, see F-09), anyone can change prices, add items or delete the catalogue. Secrets stay readable in git history after the files are deleted.
- **What to do:** load the signing key from Key Vault (the Web app already reads Key Vault in production, `src/Web/Program.cs:31-32`), with a new random key per environment. Validate issuer and audience. Cut token life to minutes (see F-13). Delete the committed key file and treat every committed value as compromised. Add secret scanning to CI. Decide with the owner whether to rewrite git history.
- **Effort:** 1.5 days · **In the recommended sprint:** yes

F-03 · The first deployment creates an administrator account with a published password · Severity: Critical · Dimension: 8

- **What we found:** On every start and in every environment, both the Web app and the API seed two accounts, a demo shopper and an administrator, with one shared default password defined in source. The login page prints both account names and the password. The constant carries the comment "TODO: Don't use this in production".
- **Evidence:** `src/ApplicationCore/Constants/AuthorizationConstants.cs:7-8`; `src/Infrastructure/Identity/AppIdentityDbContextSeed.cs:18-30`. Seeding runs outside any environment check at `src/Web/Program.cs:120-139` and `src/PublicApi/Program.cs:129-148`.

`src/Web/Areas/Identity/Pages/Account/Login.cshtml:46-54` prints the credentials. OWASP A07:2025 (default credentials).

- **Why it matters:** anyone can sign in to the production storefront as administrator and reach the admin console. If someone changed the password by hand, it stays changed. But any new environment, or a restored empty identity database, recreates the account with the published password.
- **What to do:** seed demo data only in Development. Create the first administrator in a one-off, audited step with a generated password. Remove the credentials from the login page. Check production for these accounts and disable them.
- **Effort:** 1 day · **In the recommended sprint:** yes

F-04 · The runtime reaches end of support on 10 November 2026 · Severity: High · Dimension: 2

- **What we found:** Every project targets `net8.0`, and the SDK setting, Docker images, CI, dev container and App Service runtime are all on 8.0. .NET 8 support ends on 10 November 2026, twelve days after this report. .NET 9 ends the same day, so the next supported target is .NET 10, the current LTS. The 14 ASP.NET Core and EF Core packages are pinned at 8.0.2 (February 2024), 31 patch releases behind 8.0.33.
- **Evidence:** `Directory.Packages.props:4-7`; `global.json:3`; `src/Web/Dockerfile:10,20`; `src/PublicApi/Dockerfile:3,8`; `.github/workflows/dotnetcore.yml:15`; `.devcontainer/devcontainer.json:5`; `infra/main.bicep:56-57`; full table in evidence/06.
- **Why it matters:** after 10 November, security fixes for the .NET 8 runtime and ASP.NET Core 8 stop. Security questionnaires, cyber-insurance forms and some platforms treat an unsupported runtime as an automatic fail.
- **What to do:** upgrade to .NET 10. Change the target framework, pin an exact SDK in `global.json`, and update the container images, CI and App Service runtime. Move the Microsoft packages to 10.0.x and third-party packages to versions that support .NET 10, fix breaking changes, then run the full test suite against SQL Server (F-16). As a stop-gap before the upgrade, move the 8.0.2 packages to 8.0.33 (F-05).
- **Effort:** 5 days · **In the recommended sprint:** no (Renascia). It can run in parallel with Firmora if you want both at once.

F-05 · Pinned packages carry published High-severity advisories · Severity: High · Dimension: 2

- **What we found:** Three of the 50 direct packages have advisories at the pinned version:
  - `System.Text.Json` 8.0.3: CVE-2024-30105 and CVE-2024-43485, both High (denial of service); fixed in 8.0.5.
  - `Microsoft.AspNetCore.Components.WebAssembly.Server` 8.0.2: CVE-2026-96230, High (script injection through the WebAssembly debug endpoint); fixed in 8.0.32. The app turns that endpoint on in the Development and Docker environments (`src/Web/Program.cs:169-175`).
  - `Azure.Identity` 1.10.4: CVE-2024-29992 and CVE-2024-35255, Moderate; fixed by 1.11.4. It is used on the production configuration path (`src/Web/Program.cs:31`).

  Walking the dependency metadata adds nine transitive packages with advisories (indicative, because we did not restore). The run-time ones are `AutoMapper` 12.0.1 (CVE-2026-32933, High; see F-14), `Microsoft.Extensions.Caching.Memory` 8.0.0 (CVE-2024-43483, High) and `Microsoft.Identity.Client` 4.56.0. The rest arrive through the scaffolding package, including `NuGet.Packaging` 6.3.1 (CVE-2024-0057, Critical).
- **Evidence:** `Directory.Packages.props:18,29,51`; evidence/03 (NuGet vulnerability database, 2026-10-28); evidence/04 (transitive walk, with its limits).
- **Why it matters:** denial-of-service flaws are cheap to trigger against a public shop. Any customer, insurer or penetration-test scan will list these packages. Dependabot is configured but cannot help: the repository is archived, and the last dependency merge was on 2024-04-01.
- **What to do:** now, bump within the 8.0 line: the Microsoft packages to 8.0.33, `System.Text.Json` to 8.0.6 and `Azure.Identity` to 1.21.0. Drop the scaffolding package from the runtime projects. Add `dotnet list package --vulnerable --include-transitive` to CI, failing on High or Critical. Whether a vulnerable framework-assembly copy is actually loaded depends on the runtime patch level; the CI check settles it.
- **Effort:** 1 day · **In the recommended sprint:** yes

F-06 · The sign-in cookie loses its HttpOnly flag, and password changes do not end other sessions · Severity: High · Dimension: 8

- **What we found:** The cookie setup sets `options.Cookie.HttpOnly = true`, then four lines later replaces the whole cookie builder with `new CookieBuilder { Name = ..., IsEssential = true }`. A fresh `CookieBuilder` has `HttpOnly` false and `SameSite` unspecified, so the protective defaults Identity normally supplies are lost. The same method sets `EventsType = typeof(RevokeAuthenticationEvents)`. When `EventsType` is set, ASP.NET Core resolves that type instead of the configured events, so Identity's security-stamp check (which signs other sessions out after a

password change) never runs. The custom revocation list that replaces it lives in process memory (F-18). The hardened settings at `src/Web/Program.cs:47-53` apply to a second cookie scheme, not the Identity scheme that sign-in actually uses.
- **Evidence:** `src/Web/Configuration/ConfigureCookieSettings.cs:22-34` (line 25 against line 29); `src/Web/Configuration/RevokeAuthenticationEvents.cs:23-34`; `src/Web/Areas/Identity/Pages/Account/Logout.cshtml.cs:35-40`; `src/Web/Program.cs:47-58`. The framework behaviour was checked against ASP.NET Core `release/8.0` source: the `CookieBuilder` defaults, the default `CookieAuthenticationOptions` builder (`HttpOnly = true`), `AuthenticationHandler.InitializeEventsAsync` (`EventsType` replaces `Options.Events`), and `AddIdentity` (registers `SecurityStampValidator` on `OnValidatePrincipal`). The cookie policy's `SameSite=Strict` minimum (`ConfigureCookieSettings.cs:20`) restores `SameSite` but not `HttpOnly`. OWASP A02:2025, A07:2025.
- **Why it matters:** any script-injection flaw (F-12 covers the library side) can read the session cookie and take over the account, the administrator's included. A customer who changes a stolen password stays signed in wherever the thief is.
- **What to do:** set properties on the existing builder instead of replacing it (`HttpOnly`, `SecurePolicy = Always`, `SameSite`). Have the custom events call `SecurityStampValidator.ValidatePrincipalAsync` first, or drop them and update the security stamp on logout. Add a functional test that inspects `Set-Cookie`. On a running build, confirm by reading the `Set-Cookie` header after sign-in.
- **Effort:** 1 day · **In the recommended sprint:** yes

F-07 · Deleting a product breaks the basket and checkout for every customer holding it · Severity: High · Dimension: 4

- **What we found:** Basket lines store a product id with no foreign key, and the admin API deletes products without checking baskets. Both the basket page and order creation look up each line's product with `First(...)`, which throws when the product no longer exists.
- **Evidence:** in `CatalogContextModelSnapshot.cs:52-77, 197-204`, `BasketItems` has a key and index on `BasketId` only, none on `CatalogItemId` (see also `BasketItemConfiguration.cs:9-14`). `src/PublicApi/CatalogItemEndpoints/DeleteCatalogItemEndpoint.cs:30-41` deletes without checks. The `First` lookups are at `src/Web/Services/BasketViewModelService.cs:55-60` and `src/ApplicationCore/Services/OrderService.cs:38-43`. OWASP A10:2025.
- **Why it matters:** a routine catalogue change produces errors on the basket and checkout pages, meaning lost sales and support tickets. No alert fires (F-10).
- **What to do:** agree the business rule first: mark products discontinued (soft delete), or drop missing lines from baskets with a notice. Add the foreign key or the soft-delete flag with a migration, replace `First` with a tolerant lookup, and add tests for both pages.
- **Effort:** 1.5 days · **In the recommended sprint:** yes

F-08 · The production database is reachable from almost any address, the app runs as database owner, and schema changes run at start-up · Severity: High · Dimension: 4

- **What we found:** The SQL template enables public network access and opens a firewall rule from `0.0.0.1` to `255.255.255.254`, commented as a convenience for developer debugging. The application's database user is added to `db_owner`. Both apps call `Database.Migrate()` during start-up seeding, so whichever instance starts first applies schema changes. The template sets no database tier, backup retention or long-term retention, so all three follow platform defaults we could not see. The script that creates the app user downloads Microsoft's go-sqlcmd v0.8.1 from the internet at deployment time, without a checksum, and passes it the SQL administrator password.
- **Evidence:** `infra/core/database/sqlserver/sqlserver.bicep:21-23, 28-31, 33-42, 81-95`; `src/Infrastructure/Data/CatalogContextSeed.cs:19-22`; `src/Infrastructure/Identity/AppIdentityDbContextSeed.cs:13-16`; `src/Web/Program.cs:122-139`; `src/PublicApi/Program.cs:131-148`. OWASP A02:2025, A03:2025.
- **Why it matters:** the database holds customer accounts and orders. With public exposure and owner rights, one leaked connection string gives full control (F-02 shows how secrets have been handled so far). Start-up migrations can take the site down mid-deploy, or race when two instances start together.
- **What to do:** restrict network access to a private endpoint, or to the app's outbound addresses only. Create a least-privilege app user (read and write, no schema changes) and a separate migration identity. Run migrations as a pipeline step (an EF bundle or an idempotent script). Set the tier and backup retention explicitly. Replace SQL passwords with managed identities, or at least pin and verify the downloaded tool.
- **Effort:** 2 days · **In the recommended sprint:** yes

F-09 · The deployment covers half the system, with no pipeline, staging or rollback · Severity: High · Dimension: 6

- **What we found:** The Azure Developer CLI definition deploys only the Web project. The PublicApi, which the Blazor admin console calls, is neither provisioned nor deployed, and the admin client's API address defaults to `https://localhost:5099/api/`. Deployment is run by hand from a workstation (`azd up`); CI builds and tests but never deploys. The App Service plan is B1, a Basic tier without deployment slots, so there is no staging slot and no swap-back.
- **Evidence:** `azure.yaml:4-8`; `infra/main.bicep:47-65, 120-132`; `src/BlazorAdmin/wwwroot/appsettings.json:2-5`; `README.md:96` ("the site's Admin page … must communicate with the server using the site's PublicApi"); `.github/workflows/dotnetcore.yml`.
- **Why it matters:** as inherited, either the admin console cannot work in the deployed environment, or the API is deployed by an undocumented route we cannot see. Either way, each release is a manual, unrepeatable event with no way back.
- **What to do:** add the API to the infrastructure code and `azure.yaml`, or fold its endpoints into the Web app to remove a deployable. Make the admin client's API address per-environment. Build a pipeline that builds once, deploys to staging, smoke-tests and promotes, on a tier with slots or a blue/green pair.
- **Effort:** 4 days · **In the recommended sprint:** no (Renascia)

F-10 · Production gives little visibility, and the health checks depend on a product's name · Severity: High · Dimension: 7

- **What we found:** Logging goes to the console and to App Service file logs at Verbose level, with detailed error pages. Application Insights is optional in the template and not wired up, and there are no alerts. The `/health` endpoint runs two checks that make outbound HTTP calls and report healthy only if the response contains the text ".NET Bot Black Sweatshirt", a seeded product name. The API check calls the configured API address, which defaults to localhost. Each check creates a new `HttpClient` per call.
- **Evidence:** `src/Web/Program.cs:23, 86-89, 151-168, 196-197`; `src/Web/HealthChecks/HomePageHealthCheck.cs:22-33`; `src/Web/HealthChecks/ApiHealthCheck.cs:23-31`; `infra/core/host/appservice.bicep:6, 73, 77-88`; `infra/main.bicep:47-65` (no Application Insights passed). OWASP A09:2025.
- **Why it matters:** rename or delete one product and the site reports itself unhealthy; if `/health` is ever wired to a platform probe, instances get recycled for no reason. Meanwhile real failures such as F-07 raise no alert, and Verbose logs kept for one day are both noisy and short-lived.
- **What to do:** wire Application Insights or OpenTelemetry with structured logs and request and dependency tracking. Alert on error rate, latency and failed sign-ins. Replace the checks with a liveness check and a readiness check that tests the database connection. Set log levels per environment.
- **Effort:** 2 days · **In the recommended sprint:** no (Renascia)

F-11 · Error details and developer tools are exposed outside development · Severity: Medium · Dimension: 8

- **What we found:** The API's exception middleware returns `exception.Message` to the caller for every unhandled exception, and Swagger UI is served in every environment. The Web app turns on the developer exception page, a page listing all registered services (`/allservices`), the EF migrations endpoint and WebAssembly debugging in Development and also in an environment called `Docker`, which the compose file sets.
- **Evidence:** `src/PublicApi/Middleware/ExceptionMiddleware.cs:44-51`; `src/PublicApi/Program.cs:165-173`; `src/Web/Program.cs:90-94, 169-176`; `docker-compose.override.yml:5,14`. OWASP A02:2025, A10:2025.
- **Why it matters:** internal messages help an attacker map the system. A container deployed with the compose settings runs with developer tools switched on, including the endpoint covered by CVE-2026-96230 (F-05).
- **What to do:** return standard problem details with a correlation id, and log the detail server-side. Serve Swagger only in Development or behind authentication. Remove the `Docker` special case.
- **Effort:** 0.5 days · **In the recommended sprint:** yes

F-12 · Production pages load end-of-life jQuery and Bootstrap from public CDNs, and differ from development · Severity: Medium · Dimension: 2

- **What we found:** In Staging and Production, the layout loads jQuery 2.2.0 (2016), Bootstrap JavaScript 3.3.6 (2015) and a 2016 pre-release of Bootstrap 4 CSS from public CDNs, with no Subresource Integrity on those three. Client-side validation uses jquery-validation 1.14.0 (2015). Development uses other versions (jQuery 3.6.3, Bootstrap 3.4.1). The production set carries 15 published advisories: 4 for jQuery, 7 for Bootstrap and 4 for jquery-validation, two of them High. The local fallback paths do not match where LibMan puts the files, so if a CDN fails, the fallback is likely to fail too. Two restored libraries (toastr and a deprecated SignalR client) are not used.
- **Evidence:** `src/Web/Views/Shared/_Layout.cshtml:19-24, 57-67`; `src/Web/Views/Shared/_ValidationScriptsPartial.cshtml:5-17`; `src/Web/libman.json`; evidence/05 (versions, dates, CVE list, path comparison).

- **Why it matters:** the team tests one front end and ships another, and in production the CSS and JavaScript come from different Bootstrap major versions. Any scan of the live site lists these libraries, and third-party scripts without integrity checks are a supply-chain exposure (OWASP A03:2025, A08:2025). Exploiting the jQuery and Bootstrap advisories needs untrusted HTML passed to specific functions, which we did not see in this code (`site.js` is empty), so we rate the practical risk Medium.
- **What to do:** pick one supported set (Bootstrap 5, and jQuery only where still needed) and serve it the same way in every environment, from the app or from a CDN with integrity hashes. Remove the environment split and the unused libraries, then add a visual check of the key pages.
- **Effort:** 3 days · **In the recommended sprint:** no (Renascia)

F-13 · Account security sits below today's baseline · Severity: Medium · Dimension: 8

- **What we found:**
  - E-mail sending is a stub, so confirmation e-mails are never sent, and registration signs new users in at once without requiring a confirmed account.
  - Passwords need only six characters (Identity defaults; no options are set).
  - The "Remember me" box is ignored, because the sign-in call hard-codes `isPersistent: false`. The comment above that call also describes lockout wrongly; lockout on failure is in fact on.
  - Admin-client tokens last seven days with no revocation, and the sign-in API has no rate limiting.
  - The user controller marks actions both `[Authorize]` and `[AllowAnonymous]`. An anonymous call to its logout action reaches `Claims.First(...)` and fails with a server error.
- **Evidence:** `src/Infrastructure/Services/EmailSender.cs:10-14`; `src/Web/Areas/Identity/Pages/Account/Register.cshtml.cs:50, 77-89`; `src/Web/Areas/Identity/Pages/Account/Login.cshtml.cs:74-78`; `src/Infrastructure/Identity/IdentityTokenClaimService.cs:40`; `src/PublicApi/AuthEndpoints/AuthenticateEndpoint.cs:41-44`; `src/Web/Controllers/UserController.cs:35-58`. No Identity, rate-limiter or password options are configured (evidence/07). OWASP A07:2025.
- **Why it matters:** unverified accounts and short passwords make takeover and fake sign-ups cheaper. A "Remember me" box that does nothing is a visible defect.
- **What to do:** connect a real e-mail provider and require confirmed accounts. Set a password policy (at least 12 characters). Pass `Input.RememberMe` through. Rate-limit the sign-in endpoints with the built-in rate limiter, shorten token lifetime (together with F-02), and tidy the attribute pairs.
- **Effort:** 2 days · **In the recommended sprint:** no (Renascia)

F-14 · The fix path for two libraries changes their licence · Severity: Medium · Dimension: 2

- **What we found:** From AutoMapper 15 and MediatR 13, both packages ship a licence offering the Reciprocal Public License 1.5 (RPL-1.5) or a commercial licence. The last MIT and Apache-2.0 releases are AutoMapper 14.0.0 and MediatR 12.5.0. The only fix for the AutoMapper advisory CVE-2026-32933 (High) is in 15.1.1 or 16.1.1. In this codebase, AutoMapper performs three simple mappings and MediatR dispatches two queries.
- **Evidence:** `Directory.Packages.props:19, 24`; `src/PublicApi/MappingProfile.cs:11-18` (the entire AutoMapper configuration); two `IRequestHandler` classes in `src/Web/Features/`; the licence files inside the AutoMapper 15.1.1 and MediatR 13.0.0 packages (evidence/03).
- **Why it matters:** a routine "update everything" during the upgrade would quietly bring in reciprocal or commercial licence terms. That is a decision for the owner, not a default for an engineer.
- **What to do:** replace AutoMapper with three hand-written mappings, which removes both the advisory and the licence question. Keep MediatR on 12.5.0, or replace the two handlers with direct service calls.
- **Effort:** 1 day · **In the recommended sprint:** no (Renascia)

F-15 · The catalogue API waits one second on every call and can return the whole table · Severity: Medium · Dimension: 9

- **What we found:** The paged catalogue endpoint starts with `await Task.Delay(1000)`. A missing or zero page size becomes `int.MaxValue`, so an anonymous caller can fetch every product in one response, and the admin client calls the endpoint without a page size. The storefront reads the database directly and is not slowed down; the admin console and any API consumer are. Storefront imagery is heavy: 12 product PNGs totalling 2.0 MB and a 730 KB banner. Response compression and output caching are not configured.
- **Evidence:** `src/PublicApi/CatalogItemEndpoints/CatalogItemListPagedEndpoint.cs:31-42`; `...ListPagedCatalogItemRequest.cs:12-13`; `src/ApplicationCore/Specifications/CatalogFilterPaginatedSpecification.cs:11-14`; `src/BlazorAdmin/Services/CatalogItemService.cs:84`; evidence/01 and evidence/07 (image sizes; no compression or caching calls).
- **Why it matters:** every admin page load pays a second for nothing. Unbounded queries grow with the catalogue and make denial of service cheap. Heavy images hurt mobile page speed and search ranking.

- **What to do:** delete the delay (a quick win). Cap and validate the page size. Serve images in modern formats at display sizes, and add response compression and output caching for the catalogue.
- **Effort:** 1.5 days (0.5 in Firmora, 1 in Servata) · **In the recommended sprint:** partly

F-16 · Tests do not exercise the real database, and CI is minimal and on outdated actions · Severity: Medium · Dimension: 5

- **What we found:**
  - The repository has 62 test methods across four projects and two frameworks (xUnit and MSTest).
  - Every test that touches data uses the EF Core in-memory provider, which does not enforce relational constraints, SQL translation or migrations. The SQL Server path, where F-07 and F-08 live, is untested.
  - One unit test asserts the F-01 behaviour.
  - The production API host loads `appsettings.test.json` on every start, under the comment “Use to force loading of appsettings.json of test project”.
  - CI runs build and test only, on `actions/checkout@v2` and `actions/setup-dotnet@v1`, which declare the Node 12 runtime that GitHub has retired (current majors are v7 and v6). Prerelease SDKs are allowed, and there are no coverage, vulnerability, secret or format checks.
- **Evidence:** evidence/01 (counts); `tests/FunctionalTests/Web/WebTestFixture.cs:40,48`; `tests/FunctionalTests/PublicApi/ApiTestFixture.cs:27,35`; `tests/IntegrationTests/Repositories/OrderRepositoryTests/GetByIdWithItemsAsync.cs:22`; `tests/UnitTests/MediatorHandlers/OrdersTests/GetOrderDetails.cs:22-37`; `src/PublicApi/Program.cs:30-31`; `.github/workflows/dotnetcore.yml:11-22`; evidence/06. `CodeCoverage.runsettings` excludes `moq.dll`, a library these tests do not use.
- **Why it matters:** the tests give confidence about the parts least likely to break and none about the database. CI cannot stop a vulnerable package or a committed secret.
- **What to do:** in Firmora (0.5 days), update the actions, stop allowing prerelease SDKs, and add vulnerability and secret scans. In Servata (3.5 days), run integration tests against SQL Server in a container, settle on one test framework, add a coverage report with a floor for `ApplicationCore`, and remove test configuration from the production host.
- **Effort:** 4 days (0.5 in Firmora, 3.5 in Servata) · **In the recommended sprint:** partly

F-17 · Accessibility barriers in the storefront and admin console · Severity: Medium · Dimension: front end (accessibility check)

- **What we found, by reading the markup:**
  - 19 of 20 `<img>` elements have no `alt` text, product images included.
  - The catalogue filter’s submit control is an image input with no `alt`, so it has no accessible name.
  - The brand and type selects sit in empty labels whose visible text comes from CSS `content: attr(data-title)`, which assistive technology may not announce.
  - Neither layout sets a `lang` attribute, and the admin page disables zoom.
  - The account menu (My orders, Log out) opens on mouse hover only.
  - Basket quantity inputs have no label, admin table rows respond to clicks only, and the catalogue page has no heading.
- **Evidence:** evidence/07 (static scans); `src/Web/Pages/Index.cshtml:6-20`; `src/Web/Pages/Basket/Index.cshtml:39`; `src/Web/Pages/Admin/Index.cshtml:13,16`; `src/Web/wwwroot/css/shared/components/identity/identity.scss:29-45`; `src/Web/Views/Shared/_LoginPartial.cshtml:5-31`; `src/BlazorAdmin/Pages/CatalogItemPage/List.razor:39-41`; `src/Web/wwwroot/css/catalog/catalog.component.scss:52-60`. Relevant WCAG 2.2 criteria: 1.1.1, 1.3.1, 1.4.4, 2.1.1, 2.4.6, 3.1.1, 4.1.2.
- **Why it matters:** screen-reader and keyboard users cannot reliably complete a purchase. Accessibility law in many markets covers online shops, and larger buyers ask for a conformance statement.
- **What to do:**
  - Give product images their product name as alt text; give decorative images an empty `alt`.
  - Use real labels and a `lang` attribute, and allow zoom.
  - Make the account menu a button with `aria-expanded`, and use buttons instead of clickable rows.
  - Add one `h1` per page.
  - Add automated accessibility checks of key pages to CI, then do a manual screen-reader pass.
- **Effort:** 3 days · **In the recommended sprint:** no (Renascia)

F-18 · Session revocation, caches and data-protection keys assume one instance that never restarts · Severity: Medium · Dimension: 9

- **What we found:** Logout writes a revocation entry to in-process memory, catalogue view models are cached in process, and no data-protection key store is configured. The code's own comments flag the first point ("replace IMemoryCache with a distributed cache if you are in multi-host scenario").
- **Evidence:** `src/Web/Configuration/RevokeAuthenticationEvents.cs:11, 28`; `src/Web/Areas/Identity/Pages/Account/Logout.cshtml.cs:13, 37`; `src/Web/Controllers/UserController.cs:51`; `src/Web/Services/CachedCatalogViewModelService.cs`; evidence/07 (no `AddDataProtection`).
- **Why it matters:** with two instances, or after a restart, a logged-out cookie works again. In the provided containers, the keys live inside the container, so every redeploy signs everyone out and breaks forms in flight. App Service persists keys to a shared folder, which softens this on that host. Scaling out under load would surface all of this as intermittent defects.
- **What to do:** persist data-protection keys to storage protected by Key Vault, and rely on the security stamp for revocation (F-06) or move it to a distributed cache. Keep in-process caching for data that tolerates staleness.
- **Effort:** 1.5 days · **In the recommended sprint:** no (Servata)

F-19 · Anonymous page views create database rows and a ten-year cookie, with consent switched off · Severity: Medium · Dimension: 9

- **What we found:** Opening the basket page without signing in creates a basket row in the database and sets a basket cookie that expires in ten years. The checkout page sets the same cookie without the essential flag. Nothing deletes abandoned baskets, and the cookie-consent check is commented out with a TODO.
- **Evidence:** `src/Web/Pages/Basket/Index.cshtml.cs:28-31, 93-96`; `src/Web/Services/BasketViewModelService.cs:28-51`; `src/Web/Pages/Basket/Checkout.cshtml.cs:84-96`; `src/Web/Configuration/ConfigureCookieSettings.cs:15-21`.
- **Why it matters:** crawlers that follow the basket link in every page header grow a table that is never cleaned up. A ten-year identifier with no consent mechanism is a privacy-compliance question in many markets.
- **What to do:** create a basket only when an item is added. Remove baskets idle for a set period on a schedule. Shorten the cookie's life, and agree the consent approach with the owner.
- **Effort:** 1 day · **In the recommended sprint:** no (Servata)

F-20 · Builds are not reproducible and containers are not hardened · Severity: Low · Dimension: 1

- **What we found:**
  - `global.json` asks for `8.0.x`, a form the documentation says `sdk.version` does not accept, with roll-forward to the latest feature band. CI also allows prerelease SDKs, and there are no lock files.
  - The Dockerfiles copy the whole repository before restoring, which slows builds and puts the committed key file into the build context.
  - `.dockerignore` patterns only match one directory level.
  - Containers run as root with no health check, and the API Dockerfile exposes ports the .NET 8 image does not listen on.
  - The local database image, Azure SQL Edge, was retired on 30 September 2025.
- **Evidence:** evidence/06; `global.json:3-4`; `.github/workflows/dotnetcore.yml:16`; `src/Web/Dockerfile:13-18`; `src/PublicApi/Dockerfile:5-6, 10`; `.dockerignore:7-8`; `docker-compose.yml:19`.
- **Why it matters:** two builds of the same commit can use different SDKs and package graphs, and a new developer's first `docker-compose up` depends on a retired image.
- **What to do:**
  - Pin an exact SDK, enable lock files, and run a locked-mode restore in CI.
  - Restore before copying the source, and change the `.dockerignore` patterns to `**/bin` and `**/obj`.
  - Run as the image's non-root user and add a health check.
  - Switch the local database to the SQL Server container image.
- **Effort:** 1.5 days · **In the recommended sprint:** no (Renascia)

F-21 · Layer boundaries mostly hold, but the core depends on a UI project and dead code has built up · Severity: Low · Dimension: 3

- **What we found:** The layering is clear, but `ApplicationCore` references `BlazorShared`, the admin client's DTO project, so the core cannot change without the UI's shared models. Two front ends (Razor Pages/MVC and Blazor WebAssembly) and two authentication schemes (cookies and JWT) serve a small catalogue. The basket-cookie logic is duplicated, with differences, between the basket and checkout pages. Dead or unfinished code includes:
  - a controller marked "No longer used";
  - `Microsoft.AspNetCore.Mvc` 2.2.0 declared but never referenced;
  - an unused `AUTH_KEY` constant and two unused image validators;
  - project entries for files that do not exist;

    - an order “Cancel” link to a non-existent action, which can never show because status is always “Pending”;
    - a source file whose name ends in a space.
- **Evidence:** `src/ApplicationCore/ApplicationCore.csproj:16-18`; `src/Web/Pages/Basket/Index.cshtml.cs:68-98` against `Checkout.cshtml.cs:84-96`; `src/Web/Controllers/Api/BaseApiController.cs:5`; `Directory.Packages.props:34`; `AuthorizationConstants.cs:5`; `src/PublicApi/ImageValidators.cs`; `src/BlazorAdmin/BlazorAdmin.csproj:18-28`; `src/Web/Views/Order/MyOrders.cshtml:29-31`; `src/Web/ViewModels/OrderViewModel.cs:7,12`; `src/BlazorAdmin/Services/CachedCatalogLookupDataServiceDecorator.cs`.
- **Why it matters:** each extra moving part costs time on every change and every upgrade, and dead code misleads new developers.
- **What to do:** move what the core needs out of `BlazorShared`, delete the dead code, and put basket identity handling in one service. Decide whether the admin console needs its own API and token scheme.
- **Effort:** 2 days · **In the recommended sprint:** no (Servata)

F-22 · Error-prone patterns in a few places · Severity: Low · Dimension: 3

- **What we found:**
    - The seeding retry catches an exception, retries recursively, then rethrows the original exception even when the retry succeeded, with no delay between attempts.
    - An `async void` click handler means exceptions bypass the caller.
    - The admin `HttpService` returns `null` on any failed response, and callers dereference the result.
    - A `new HttpClient()` is created per health-check call.
    - Nullable reference types are off in three projects.
    - Page counts are computed with `int.Parse(Math.Ceiling(...).ToString())`.
- **Evidence:** `src/Infrastructure/Data/CatalogContextSeed.cs:48-56`; `src/BlazorAdmin/Pages/CatalogItemPage/List.razor.cs:43`; `src/BlazorAdmin/Services/HttpService.cs:29-31, 40-44` and `CatalogItemService.cs:37, 42`; `src/Web/HealthChecks/*.cs:24-25`; evidence/01; `CatalogItemListPagedEndpoint.cs:64`; `src/Web/Services/CatalogViewModelService.cs:70`.
- **Why it matters:** these patterns turn transient faults, such as a database that starts slowly, into failures that are hard to trace.
- **What to do:** rely on EF Core’s retry-on-failure for start-up, use `async Task` handlers, return results instead of `null`, create clients through `IHttpClientFactory`, and enable nullable checking everywhere.
- **Effort:** 1 day · **In the recommended sprint:** no (Servata)

F-23 · Host filtering is not configured, and a health check fetches a URL built from the request’s Host header · Severity: Low · Dimension: 8

- **What we found:** `AllowedHosts` sits inside the `Logging` section of `appsettings.json`, so host filtering receives no list. The home-page health check builds its target URL from the incoming request’s scheme and `Host` header and fetches it from the server.
- **Evidence:** `src/Web/appsettings.json:11-19` (`AllowedHosts` at line 18, inside `Logging`); `src/Web/HealthChecks/HomePageHealthCheck.cs:22-26`. OWASP A01:2025 maps server-side request forgery (CWE-918) here.
- **Why it matters:** a request with a forged `Host` header can make the server call an address of the sender’s choosing. How far that goes depends on the hosting front end; App Service routes by host name, which limits it. The likelihood is low and the fix is easy.
- **What to do:** move `AllowedHosts` to the configuration root and list the real host names. Stop health checks calling back over HTTP (F-10).
- **Effort:** 0.5 days · **In the recommended sprint:** no (Renascia, with F-10)

F-24 · Operational knowledge is not written down, and upstream is archived · Severity: Low · Dimension: 10

- **What we found:** The README explains local setup well (LocalDB, Docker, dev container, `azd`). There is no deployment runbook, no account of how the API is deployed, no decision records, no security policy and no restore procedure. The upstream repository was archived on 13 January 2025 and points to a successor sample, so no upstream fixes will come.
- **Evidence:** `README.md:47-165`; evidence/01 (repository documents); evidence/00 (archive date, last code change 2024-04-01).
- **Why it matters:** whoever inherits this owns every future fix. Without a runbook, the first incident or restore is improvised.
- **What to do:** write a one-page runbook (deploy, roll back, restore, rotate secrets), a short architecture note recording the decisions taken in the sprints, and a `SECURITY.md` with a contact route.

- **Effort:** 1.5 days · **In the recommended sprint:** no (Servata)

5. Risk register

#	Risk	Likelihood	Impact	Early warning	Mitigation	Owner
R1	Customer order data read by other customers (F-01)	H	H	Sequential requests to the order-detail address in access logs	Ownership check and test (Firmora)	Mervenor (fix); client (breach assessment)
R2	Catalogue changed or wiped through forged admin tokens (F-02)	M (needs the API deployed)	H	Unexpected catalogue edits; API calls with tokens not issued by the app	Key in Key Vault, rotation, issuer and audience checks (Firmora)	Mervenor
R3	Administrator account taken over with the published password (F-03)	H if deployed as is	H	Admin sign-ins from unknown addresses	Remove seeding outside Development; disable the seeded accounts (Firmora)	Client (account review); Mervenor (code)
R4	Runtime unsupported from 10 Nov 2026; no security patches (F-04)	H (certain)	H	Microsoft security advisories naming .NET 8 after that date	Patch bump now (Firmora), .NET 10 upgrade (Renascia)	Mervenor
R5	Known package flaws exploited for denial of service (F-05)	M	M	Spikes of failed requests, process restarts	Patch bump and CI vulnerability gate (Firmora)	Mervenor
R6	Session cookies stolen through a script flaw; stolen sessions survive password changes (F-06)	L–M	H	Concurrent sessions for one account from distant addresses	Restore HttpOnly and security-stamp validation (Firmora)	Mervenor
R7	Basket and checkout errors after a product is removed (F-07)	H	M	Rising 500s on basket and checkout (not visible today, F-10)	Soft delete or foreign key, tolerant lookups (Firmora)	Client (rule); Mervenor (code)
R8	Database compromised or damaged through open network access and owner rights; failed start-up migration (F-08)	M	H	Failed sign-ins on the SQL server from unknown addresses; slow start-ups during deploys	Network restriction, least privilege, migrations in the pipeline (Firmora)	Mervenor; client (cloud access)
R9	Failed or partial release with no way back; admin console unusable (F-09)	H	M	Releases needing manual fixes; admin console errors	Pipeline, staging, API in infrastructure code (Renascia)	Mervenor
R10	Outages or attacks go unnoticed (F-10)	H	M	Customers reporting problems before monitoring does	Telemetry, alerts, real health checks (Renascia)	Mervenor; client (on-call)
R11	Licence obligations added by an unreviewed upgrade (F-14)	M	M	Package licence changes in update pull requests	Replace AutoMapper; hold MediatR at 12.5.0 (Renascia)	Client (decision); Mervenor
R12	Accessibility complaint or failed buyer assessment (F-17)	M	M	Complaints; procurement questionnaires	Fixes plus automated checks (Renascia)	Client (statement); Mervenor (code)

6. Prioritised plan

All sizes are **engineering days** for an engineer familiar with ASP.NET Core. There are no prices in this sample.

6.1 Quick wins (each under a day; counted inside the findings' effort)

#	Action	Finding	Days
1	Scope order detail to the signed-in buyer; fix the unit test	F-01	0.5
2	Stop seeding outside Development; remove credentials from the login page	F-03	0.25
3	Set cookie properties on the existing builder instead of replacing it	F-06	0.25

4	Bump `System.Text.Json`, `WebAssembly.Server`, `Azure.Identity` and the 8.0.2 line to current patches	F-05	0.5
5	Return problem details instead of exception text; Swagger in Development only	F-11	0.5
6	Delete `Task.Delay(1000)`; cap the page size	F-15	0.25
7	Update GitHub Actions to current majors; stop allowing prerelease SDKs	F-16	0.25
8	Pass “Remember me” through to sign-in	F-13	0.1
9	Move `AllowedHosts` to the configuration root and list the real host names	F-23	0.1
10	Add `lang`, and `alt` text on product images	F-17	0.5

Items 8–10 belong to Renascia findings and can be pulled forward at no extra cost.

6.2 Sprints

Phase	Goal	Work items (finding refs)	Engineering days
Stabilise · Firmora sprint	Close the exploitable gaps; make the database and deploys safe; add tests where change is needed	F-01 (0.5), F-02 (1.5), F-03 (1), F-05 (1), F-06 (1), F-07 (1.5), F-08 (2), F-11 (0.5), F-15 part (0.5), F-16 part (0.5)	**10** (+1.5 contingency)
Upgrade · Renascia sprints 1–2	Move to .NET 10 and supported libraries; deploy the whole system through a pipeline; see what production is doing	F-04 (5), F-09 (4), F-10 (2), F-12 (3), F-13 (2), F-14 (1), F-17 (3), F-20 (1.5), F-23 (0.5)	**22** (+4 contingency)
Servata retainer	Keep it healthy; ship small improvements; pay down structure and test debt	F-15 rest (1), F-16 rest (3.5), F-18 (1.5), F-19 (1), F-21 (2), F-22 (1), F-24 (1.5)	**11.5**, spread over roughly three months
Total		24 findings	**43.5** (+5.5 contingency = 49)

6.3 Order and dependencies

- Firmora comes first, because F-01 to F-03 are exploitable today.
- The F-05 patch bump is a stop-gap only. No further 8.0 patches will be published after 10 November 2026, so the F-04 upgrade should start straight after Firmora, or alongside it if you want both at once.
- F-14 must be decided before F-04 updates third-party packages.
- F-10 depends on F-09’s pipeline for its alert configuration.
- F-16’s move to SQL Server-backed tests is easier after F-08 separates the migration identity.

---

7. Recommended sprint: scope, exclusions, assumptions (Firmora, 10 engineering days)

- **In scope:**
  - Order-detail ownership check (F-01).
  - Signing key and secrets out of source, with rotation and token validation (F-02).
  - No seeded accounts outside Development (F-03).
  - Patch-level package updates and a CI vulnerability gate (F-05).
  - Cookie hardening and security-stamp validation restored (F-06).
  - Safe product removal (F-07).
  - Database network restriction, least-privilege user, and migrations moved out of start-up (F-08).
  - Safe error responses, with developer tools in Development only (F-11).
  - Catalogue API delay removed and page size capped (F-15, part).
  - CI actions updated, with secret and vulnerability scans (F-16, part).
- **Acceptance tests:**
  1. A signed-in user who requests another user’s order number gets “not found”, proven by an automated test with two accounts.
  2. No key, password or connection-string literal remains at HEAD, and CI secret scanning passes. The API signing key comes from Key Vault, tokens signed with the old key are rejected, and issuer and audience are validated.

3. A fresh non-Development environment starts with no demo or administrator accounts, and the login page shows no credentials.
4. The sign-in `Set-Cookie` header carries `HttpOnly`, `Secure` and `SameSite`, and a password change signs out the account's other sessions. Both are proven by tests.
5. Removing a product that sits in a basket leaves the basket page and checkout working, proven by tests.
6. The database accepts connections only from the application's network path, and the app's database user cannot alter the schema. Migrations run as a pipeline step; the application no longer migrates at start-up.
7. `dotnet list package --vulnerable --include-transitive` reports no High or Critical runtime packages, and CI fails if one appears.
8. API errors return problem details without exception text, and Swagger and the developer middleware are absent outside Development.
9. The catalogue API answers without the artificial delay and rejects page sizes above the agreed cap.

- **Not included:** third-party costs (hosting, licences, AI usage); the .NET 10 upgrade (Renascia); features not listed; data migration; rewriting git history (an owner decision, scoped separately if wanted); code outside this repository; out-of-hours support.
- **Assumptions:** repository and cloud access within 3 working days; a staging environment exists or is created in the sprint; decisions within 2 working days (notably the product-removal rule in F-07); the client rotates any production credentials we identify.
- **Contingency:** up to 1.5 engineering days for named unknowns: checking production logs for signs that F-01 to F-03 were exploited, and any data clean-up that follows. Used only with written approval.

8. What we did not assess, and why

This is a desk review of public source code, so several things a full Perlustra audit covers were out of reach by design.

- **Nothing was built, restored, run or tested.** We cannot say whether the solution compiles, whether the 62 tests pass, or what they cover. A full audit runs `dotnet build`, `dotnet test` with coverage, and `dotnet list package --vulnerable --include-transitive` on a clean checkout.
- **No runtime behaviour.** Findings that describe behaviour (F-06 cookies, F-07 exceptions, F-12 fallbacks, F-13 "Remember me") are read from code and, where noted, from the ASP.NET Core 8.0 source. Each has a one-line check on a running build, listed in the finding.
- **No performance numbers or page-speed scores.** These need the site running under load and a lab measurement on key pages.
- **No automated accessibility scan or search crawl.** Both need rendered pages; our accessibility points come from reading markup and styles.
- **No production environment.** We saw no cloud configuration, logs, data, costs, backups or deployment history, so where the code relies on platform defaults (database tier, backup retention), we say so rather than guess.
- **The transitive dependency list is approximated** from package metadata, not resolved by a restore (evidence/04).
- **Git history** was read for metadata only (dates, counts). We did not scan old commits for secrets; a full audit does.
- **No people.** A client audit includes interviews about history, pain points and what only one person knows. A sample has none.

Appendix A — Dependencies and end-of-life dates

Component	Current version	Latest supported	End of support	Known vulnerabilities	Action
.NET runtime and SDK	8.0 (`net8.0`; SDK `8.0.x`)	10.0 (LTS)	10 Nov 2026	Depends on installed patch level (not visible in code)	Upgrade to .NET 10 (F-04)
ASP.NET Core and related packages (11)	8.0.2	8.0.33 / 10.0.14	10 Nov 2026	1 High (`Components.WebAssembly.Server`)	Patch now, then 10.0.x (F-05, F-04)
EF Core packages (3)	8.0.2	8.0.33 / 10.0.14	10 Nov 2026	0 direct; 1 High transitive (`Caching.Memory` 8.0.0)	As above
`System.Text.Json`	8.0.3	8.0.6 / 10.0.14	10 Nov 2026 (8.x)	2 High	Patch now (F-05)
`Azure.Identity`	1.10.4	1.21.0	–	2 Moderate	Update (F-

					05)
`AutoMapper` (via DI package 12.0.1)	12.0.1	16.2.0 (new licence terms)	–	1 High	Remove (F-14)
`MediatR`	12.0.1	12.5.0 (last Apache-2.0); 14.2.0	–	0	Hold at 12.5.0 or remove (F-14)
`Swashbuckle.AspNetCore` (3 packages)	6.5.0	6.9.0 / 10.3.1	–	0	Update with .NET 10
`System.IdentityModel.Tokens.Jwt`	7.3.1	8.23.0 (marked legacy on NuGet)	–	0	Update with .NET 10
`Microsoft.AspNetCore.Mvc`	2.2.0 (2018)	–	ASP.NET Core 2.2 line long unsupported	0	Delete; unreferenced (F-21)
`BlazorInputFile`	0.2.0 (2020)	0.2.0 (no newer release)	–	0	Replace with the built-in `InputFile` component
jQuery (production CDN / development)	2.2.0 / 3.6.3	4.0.0	2.x end of life	4 / 0	One supported version everywhere (F-12)
Bootstrap JS / CSS (production)	3.3.6 / 4.0.0-alpha.5	5.3.8	3.x: 2019-07-24; 4.x: 2023-01-01	7 (3.3.6)	Bootstrap 5 (F-12)
jquery-validation (production / development)	1.14.0 / 1.19.5	1.22.1	–	4 (2 High) / 1	Update (F-12)
Test stack	xUnit 2.7.0 (marked legacy, successor xunit.v3); MSTest 3.2.2	xunit.v3; MSTest 4.5.2	–	0	One framework (F-16)
CI actions	`checkout@v2`, `setup-dotnet@v1` (Node 12)	v7, v6	Node 12 retired for actions	–	Update (F-16)
Local database image	Azure SQL Edge	SQL Server container image	Retired 30 Sep 2025	–	Replace (F-20)

The full list of all 50 declared packages, with publish dates, latest versions, deprecation flags and advisories, is in `evidence/03-nuget-direct-packages.md` and `evidence/nuget-packages.tsv`. The transitive walk is in `evidence/04-transitive-advisories.md`. Front-end libraries, with CVE identifiers, are in `evidence/05-front-end-libraries.md`.

Appendix B — Test and quality baseline

Layer	Project	Framework	Files	Test methods	Database used
Unit	UnitTests	xUnit 2.7.0	23	36	none (substitutes)
Integration	IntegrationTests	xUnit 2.7.0	3	3	EF Core in-memory
Functional	FunctionalTests	xUnit 2.7.0	12	13	EF Core in-memory
API integration	PublicApiIntegrationTests	MSTest 3.2.2	7	10	EF Core in-memory (`appsettings.test.json`)
Total			**45**	**62**	

- **Size:** 1,689 non-blank lines of test C# against 5,253 of application C#, a ratio of 0.32. Counts come from the source; nothing was run.
- **Coverage:** unknown, because nothing was run. `coverlet.collector` is referenced by one project only.
- **CI status:** a build-and-test workflow exists (evidence/06). We could not see run history from our environment.
- **Lint and type checks:** a 150-line `.editorconfig`, not enforced in the build (no `EnforceCodeStyleInBuild`, no `TreatWarningsAsErrors`). Nullable reference types are enabled in 7 of 10 projects.
- **How to reproduce** (commands we did not run): `dotnet test eShopOnWeb.sln --collect:"XPlat Code Coverage"`; `dotnet format eShopOnWeb.sln --verify-no-changes`; `dotnet list eShopOnWeb.sln package --vulnerable --include-transitive`.

Appendix C — Environments

Environment	Hosting	How it is deployed	Backups	Monitoring	Access notes
Production (as defined in `infra/`)	App Service (Linux, B1, `dotnetcore 8.0`), two Azure SQL servers, Key Vault	`azd up` from a workstation; Web app only (F-09)	Not defined in code; platform defaults apply	App Service file logs (Verbose, 1-day retention); Application Insights not wired (F-10)	Key Vault access policies for the app's managed identity and the deploying principal; SQL firewall open to almost all addresses (F-08)
Staging	None defined	–	–	–	B1 offers no deployment slots
Local	LocalDB, `docker-compose` (Web, PublicApi, Azure SQL Edge), dev container (.NET 8), or the in-memory database	`dotnet run` / `docker-compose up`	–	Console	`ASPNETCORE_ENVIRONMENT=Docker` enables developer middleware (F-11); fixed `sa` password (F-02)

Appendix D — Method

- **Our five-day method:** day 1, intake and inventory; day 2, dependencies and runtime; day 3, security and data; day 4, delivery, operations and front end; day 5, report and sprint scope. For this sample we ran days 1–5 as a desk review only. Every step that needs a build, a running site, production access or interviews is listed in section 8.
- **Source:** a shallow clone of `dotnet-architecture/eShopOnWeb` at `4da8212117e87d808d4bbc7da6286fd2147ce606` (2025-01-13), read only. A blobless clone was used for commit metadata. The clone is not included in this deliverable.
- **Tools:** git 2.43.0; Python 3.13 standard-library scripts for parsing and counting (no project code imported or executed); pygount 3.2.0; the NuGet V3 API and NuGet vulnerability database; OSV.dev (GitHub Advisory Database records); the npm registry and the cdnjs API; endoflife.date; ASP.NET Core `release/8.0` source; OWASP Top 10:2025; Microsoft Learn. Details are in `evidence/00-source-and-method.md`.
- **AI assistance:** an AI assistant read the source, ran the metadata lookups and drafted this report. Every finding cites the files and lines it rests on, and framework behaviour was checked against the framework's source. In a client engagement, a Mervenor engineer re-checks every finding before delivery.
- **Data:** no production data exists for a sample, and none was accessed. Secrets visible in the public repository are redacted in our excerpts.

Appendix E — Evidence index

File	Contents
`evidence/00-source-and-method.md`	Pinned commit, repository status and history, fetch method, tools, what was not done
`evidence/01-inventory-and-counts.md`	Projects, line counts, test counts, other inventory facts
`evidence/02-code-excerpts.md`	69 excerpts with original line numbers, tagged by finding; secrets redacted
`evidence/03-nuget-direct-packages.md`	All 50 declared NuGet packages: pinned, latest, deprecation, advisories
`evidence/04-transitive-advisories.md`	Indicative transitive walk (277 packages) and the 9 with advisories
`evidence/05-front-end-`	jQuery, Bootstrap and validation versions by environment, CVEs, fallback paths

`libraries.md`	
`evidence/06-runtime-ci-and-containers.md`	Where .NET 8 is set; CI, Dependabot, Dockerfiles, compose, Bicep
`evidence/07-static-scans.md`	Text searches: images without `alt`, CDN references, TODOs, injection sinks, missing configuration, unreferenced packages, image weights
`evidence/nuget-packages.tsv`, `transitive-resolved.tsv`, `transitive-vulnerable.json`	Raw machine-readable lists behind 03 and 04

Perlustra is Mervenor's fixed-scope codebase health audit. Want this for your app? Mervenor — hello@mervenor.com — https://mervenor.com